

LoanLens

Loan Default Prediction using Machine Learning

Final Project Report — Phase 6.1 to End

Darshan University — Computer Engineering Department

ML Project — Academic Year 2025–26

Report Scope

This single consolidated report documents the project from introduction and dataset analysis through Machine Learning, evaluation, visualization, Flask inference, frontend integration, backend architecture, deployment, final results, conclusion, and evaluation readiness.

Project Status: End-to-End application deployed and verified

Table of Contents

Section	Title
6.1	Project Introduction & Problem Statement
6.2	Dataset Description, Data Preprocessing & EDA
6.3	Machine Learning Models & Scratch Implementation
6.4	Model Evaluation
6.5	Advanced Training, Cross-Validation & Hyperparameter Tuning
6.6	Visualization of Metrics & Graphs
6.7	Flask Project Setup & ML Inference Backend
6.8	Frontend Development & User Interface
6.9	Backend Integration, GitHub & Deployment
6.10	Final Evaluation, Results, Conclusion & Future Scope

6.1 Project Introduction & Problem Statement

6.1.1 Project Introduction

LoanLens is a Machine Learning based loan default prediction application. The project treats loan default prediction as a binary classification problem using borrower financial, demographic, employment, and loan-related information.

The system follows an end-to-end workflow: dataset exploration, data cleaning, preprocessing, EDA, model training, evaluation, model comparison, inference API development, frontend integration, and cloud deployment.

6.1.2 Problem Statement

To develop a Machine Learning classification system that predicts whether a borrower will default on a loan based on their financial, demographic, employment, and loan-related attributes.

6.1.3 Objective

- Analyze the loan dataset and understand borrower characteristics.
- Clean and preprocess numerical and categorical data.
- Train multiple classification algorithms.
- Implement at least one algorithm from scratch without a Machine Learning library.
- Evaluate and compare model performance using suitable metrics.
- Perform hyperparameter tuning and cross-validation.
- Visualize model and dataset statistics.
- Build a Flask inference service and integrate it with the web application.
- Deploy the complete application.

6.1.4 Target Variable

Value	Meaning
0	No Default
1	Default

6.1.5 Dataset Overview

Property	Value
Dataset	Loan Default Prediction
Domain	Banking
Source	Kaggle
Rows	255,347
Columns	18
Problem Type	Binary Classification

6.2 Dataset Description, Data Preprocessing & EDA

6.2.1 Dataset Features

Numerical features: Age, Income, LoanAmount, CreditScore, MonthsEmployed, NumCreditLines, InterestRate, LoanTerm, DTIRatio.

Categorical features: Education, EmploymentType, MaritalStatus, HasMortgage, HasDependents, LoanPurpose, HasCoSigner.

Identifier: LoanID — removed before model training.

6.2.2 Data Cleaning

- Missing values were checked: 0 missing values.
- Duplicate rows were checked: 0 duplicate rows.
- LoanID was removed as an identifier.
- Default was verified as the binary target.

6.2.3 Target Distribution

Class	Count	Percentage (approx.)
0 — No Default	225,694	88.4%
1 — Default	29,653	11.6%

The target is imbalanced. Consequently, Accuracy alone can be misleading and class-sensitive metrics such as Recall, Precision, F1-score and ROC-AUC are important.

6.2.4 Train/Test Split

An 80/20 stratified split was used: 204,277 training observations and 51,070 test observations.

6.2.5 Preprocessing Pipeline

Component	Method	Features
Numerical	StandardScaler	9
Categorical	OneHotEncoder(drop='first')	7
Final feature matrix	Encoded numerical representation	24

6.2.6 EDA

- Target distribution analysis was performed.
- Numerical distributions were inspected.
- Boxplots were used to inspect numerical spread and potential outliers.
- Categorical distributions were examined.
- EDA and model-evaluation visualizations were generated; the project audit records 14 plot assets.

6.2.7 Final Data Flow

**Raw Dataset → Cleaning → LoanID Removal → Stratified Split → StandardScaler + OneHotEncoder
→ 24 Features → Model Training**

6.3 Machine Learning Models & Scratch Implementation

6.3.1 Models Used

Model	Main Idea
Logistic Regression	Linear probabilistic classifier for binary classification
KNN	Distance-based classification using neighboring observations
Naive Bayes	Probabilistic classification using Bayes' theorem
Decision Tree	Recursive feature splitting using tree-based rules

6.3.2 Production Model Configurations

Model	Production configuration
Logistic Regression	C=0.1; solver=lbfgs; class_weight=balanced; max_iter=1000
KNN	n_neighbors=25; weights=distance; metric=minkowski
Naive Bayes	var_smoothing=1e-5
Decision Tree	criterion=entropy; max_depth=6; min_samples_leaf=50; min_samples_split=20

6.3.3 Logistic Regression from Scratch

A Logistic Regression implementation was created using NumPy without sklearn's LogisticRegression estimator. The implementation contains the mathematical learning steps explicitly.

- Initialize weights and bias.
- Compute the linear combination $Xw + b$.
- Apply the sigmoid function.
- Compute Binary Cross-Entropy loss.
- Compute gradients for weights and bias.
- Update parameters using batch Gradient Descent.
- Repeat for the training iterations.
- Generate probability and class predictions.

6.3.4 Scratch Implementation Result

Measure	Scratch Logistic Regression
Implementation	Pure NumPy batch Gradient Descent
ROC-AUC	0.7530
Library Logistic Regression ROC-AUC	0.7532
Weight Pearson correlation	0.9946

The scratch implementation produced a ROC-AUC very close to the library Logistic Regression model, providing a practical verification of the independently implemented learning procedure.

6.4 Model Evaluation

6.4.1 Evaluation Metrics

Metric	Description
Accuracy	Fraction of all test observations classified correctly.
Precision	Fraction of predicted Default cases that are actually Default.
Recall	Fraction of actual Default cases detected.
F1-score	Harmonic mean of Precision and Recall.
ROC-AUC	Discrimination/ranking ability across thresholds.

6.4.2 Final Untouched Test-Set Benchmark

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train time
Logistic Regression	0.6764	0.2196	0.6995	0.3342	0.7532	3.95 s
KNN	0.8851	0.5455	0.0070	0.0137	0.6927	38.33 s
Naive Bayes	0.8849	0.6080	0.0256	0.0492	0.7499	0.99 s
Decision Tree	0.8849	0.5888	0.0302	0.0574	0.7254	44.16 s

The benchmark shows a strong effect of class imbalance: several models have high Accuracy while detecting very few Default cases. Logistic Regression has substantially higher Default Recall and the highest ROC-AUC among the four production benchmarks.

6.4.3 Confusion-Matrix Interpretation

The project evaluation includes confusion matrices for the four models. The confusion matrices expose the difference between correctly identified Default cases and Default cases classified as No Default, which is not visible from Accuracy alone.

6.5 Advanced Training, Cross-Validation & Hyperparameter Tuning

6.5.1 Hyperparameter Tuning

The academic notebook includes GridSearchCV-based tuning for the four models. The notebook's Week 4 tuning record is retained here as the documented experiment record.

Model	Notebook/PDF tuning result
Logistic Regression	C=0.01; solver=liblinear; CV=3; scoring=accuracy
KNN	n_neighbors=15; weights=uniform; tuning on 30,000 training samples; CV=3
Naive Bayes	var_smoothing=0.01; CV=3
Decision Tree	max_depth=5; min_samples_leaf=5; min_samples_split=10

Documentation note: The notebook/PDF tuning settings above and the current production artifact settings are different experiment records. The deployed application uses the production artifact configurations documented in Section 6.3.2; no silent replacement of the notebook results is made.

6.5.2 Cross-Validation

The production benchmark includes 5-fold cross-validation using ROC-AUC as the validation measure.

Model	5-fold CV ROC-AUC	Std. Dev.
Logistic Regression	0.7426	0.0063
KNN	0.6753	0.0068
Naive Bayes	0.7396	0.0044
Decision Tree	0.7036	0.0057

Cross-validation provides an additional view of model stability across folds and complements the final untouched test-set evaluation.

6.6 Visualization of Metrics & Graphs

The project includes visualization of dataset characteristics and model performance. The project audit records 14 EDA and model-evaluation plot assets in the public artifacts area.

6.6.1 Dataset Visualizations

- Target-class distribution.
- Numerical feature distributions.
- Boxplots for numerical feature spread and potential outliers.
- Categorical distributions.
- Feature correlation analysis.

6.6.2 Model Visualizations

- Confusion matrices.
- ROC curves and ROC-AUC comparison.
- Model metric comparison.
- Scratch Logistic Regression loss curve.
- Model benchmark and cross-validation summaries.

6.6.3 Scratch Gradient Descent Visualization

The notebook records a pure NumPy Gradient Descent implementation trained for 1,000 epochs. The documented loss decreased from 0.693147 at initialization to 0.3438417, with a recorded scratch accuracy of 0.8838653 in the notebook experiment.

These notebook values are kept as the original scratch-experiment record and are separate from the production benchmark in Section 6.4.

6.7 Flask Project Setup & ML Inference Backend

6.7.1 Flask Architecture

The ML inference engine is implemented as a Python Flask service. The Flask service loads the trained preprocessor and model artifacts once during startup and serves inference requests without retraining on each request.

6.7.2 Loaded Artifacts

Artifact	Purpose
preprocessor.pkl	ColumnTransformer: StandardScaler + OneHotEncoder, producing 24 encoded features
logistic_regression.pkl	Production Logistic Regression model
knn.pkl	Production KNN model
naive_bayes.pkl	Production Gaussian Naive Bayes model
decision_tree.pkl	Production Decision Tree model

6.7.3 Main API Endpoints

Method	Endpoint	Purpose
GET	/api/health	Service and model health check
POST	/api/predict	Borrower prediction/inference
GET	/api/models	List four primary production models
GET	/api/models/comparison	Model metrics, confusion matrices and ROC data
GET	/api/models/<modelId>	Per-model parameters and explanation data
GET	/api/data/insights	EDA statistics and dataset insights
POST	/api/simulate	Risk sensitivity simulation

6.7.4 Inference Flow

User Input → Node/Express API Gateway → Flask /api/predict → Preprocessor → Logistic Regression → Probability/Classification → Risk presentation → Frontend

6.8 Frontend Development & User Interface

6.8.1 Frontend Technology

The deployed frontend uses server-side rendered EJS views on Node.js/Express, with Vanilla JavaScript and custom CSS. It is not a React/Vite frontend.

6.8.2 Main Pages

Route	Purpose
/overview or /dashboard	Project overview and model dashboard
/assess or /predict	Borrower risk assessment form
/model/context	ML pipeline and domain context
/model/comparison	Four-model comparison and ROC information
/data/insights	Dataset EDA and statistical insights
/model/details	Per-model details and explanations
/predictions	Prediction/history interface
/simulator	Interactive sensitivity simulation

6.8.3 Frontend Integration

- predictionService routes assessment requests to the prediction API.
- modelService consumes model metadata and comparison endpoints.
- analyticsService consumes dataset insight data.
- Real API-backed pages were verified with HTTP 200 responses.
- The existing LoanLens design system was preserved during backend integration.

6.8.4 User Result Presentation

The application presents the prediction probability, risk score, risk level, model used, and supporting factor information in an educational ML interface. The academic presentation uses terms such as "Predicted Default Risk" and "Model Classification" rather than treating the prediction as a real banking approval decision.

6.9 Backend Integration, GitHub & Deployment

6.9.1 Backend Architecture

LoanLens uses a dual-layer architecture: Node.js/Express provides server-side rendering and the API gateway, while Python Flask provides the ML inference engine. Express communicates with Flask over the internal loopback address 127.0.0.1:5001.

6.9.2 Production Container

- Unified Docker container using Python 3.11 and Node.js 20.x.
- TypeScript is compiled to dist/ and production starts with node dist/server.js.
- Flask runs internally on 127.0.0.1:5001.
- Render supplies the public PORT and the Node server binds to 0.0.0.0.
- ML .pkl artifacts, views, static assets and source files are preserved in the Docker build context.

6.9.3 Deployment Files

File	Role
Dockerfile	Unified Node + Python production image
.dockerignore	Controls Docker build context while preserving required artifacts
render.yaml	Render Blueprint / infrastructure configuration
.env.example	Documented environment configuration template

6.9.4 GitHub Repository

Repository: github.com/Bhaviik-5202/LoanLens

The project was committed and pushed to the main branch after correcting the initial Git initialization/staging issue.

6.9.5 Render Deployment

Live application URL: <https://loanlens-hynm.onrender.com>

The deployment initially exposed a scikit-learn artifact/runtime version mismatch. The runtime was aligned to scikit-learn 1.6.1, after which the preprocessing pipeline and all four model artifacts loaded successfully.

6.9.6 Production Verification

Verification	Result
GET /	200 OK
GET /overview	200 OK
GET /assess	200 OK
GET /model/context	200 OK
GET /model/comparison	200 OK
GET /data/insights	200 OK
GET /simulator	200 OK
GET /api/health	200 OK
GET /api/models	200 OK
GET /api/models/comparison	200 OK
GET /api/data/insights	200 OK
POST /api/predict	200 OK
Node → Flask communication	Verified

6.9.7 Example Production Prediction

Input	Example
Age	42
Income	115,000
Loan Amount	25,000
Credit Score	780
Months Employed	64
Interest Rate	6.8%
DTI Ratio	0.22
Model	Logistic Regression
Default Probability	0.1148
Prediction	0 — No Default
Risk Score	89
Risk Level	Low

This is an application test case and should be interpreted as a Machine Learning prediction output, not as a real lending approval or financial decision.

6.10 Final Evaluation, Results, Conclusion & Future Scope

6.10.1 SOP Alignment

SOP Week	Requirement	Project Evidence / Status
1	Problem definition & dataset exploration	Completed
2	Cleaning, preprocessing, EDA	Completed
3	Model creation + scratch algorithm	Completed; 4 models + scratch LR
4	Model evaluation	Completed; final test metrics
5	Advanced training, CV, comparison, tuning	Completed/documented; production CV + notebook tuning record
6	Metric visualizations	Completed; EDA/model plots and scratch loss
7	Flask setup	Completed
8	Frontend	Completed
9	Backend + deployment	Completed; GitHub + Render
10	Project evaluation	Application/report prepared for evaluation; viva preparation remains an evaluation activity

6.10.2 Final Model Benchmark Summary

Model	Test ROC-AUC	5-Fold CV AUC
Logistic Regression	0.7532	0.7426 ± 0.0063
KNN	0.6927	0.6753 ± 0.0068
Naive Bayes	0.7499	0.7396 ± 0.0044
Decision Tree	0.7254	0.7036 ± 0.0057

For the deployed production application, Logistic Regression is the selected model. The selection is based on the documented production benchmark, particularly its ROC-AUC and substantially higher Default Recall under the imbalanced target distribution.

6.10.3 Technical Achievements

- Built a complete binary-classification ML pipeline.
- Processed mixed numerical and categorical borrower data into 24 model features.
- Implemented Logistic Regression from scratch using NumPy.
- Trained and evaluated four primary classification models.
- Performed hyperparameter tuning and 5-fold cross-validation.
- Created EDA and model-performance visualizations.
- Built a Flask ML inference backend.
- Integrated Flask with a Node.js/Express SSR application.
- Containerized the application using Docker.

- Deployed the application on Render.
- Verified production endpoints and end-to-end prediction flow.

6.10.4 Limitations

- The dataset is imbalanced, which affects the interpretation of Accuracy.
- The model is trained on the available Kaggle dataset and may not represent every real-world lending population.
- Model predictions should be treated as educational/analytical outputs rather than guaranteed financial outcomes.
- The notebook tuning record and deployed production artifact settings are separate experiment records and should be identified accordingly.

6.10.5 Future Scope

- Evaluate additional classification algorithms and calibrated probability methods.
- Explore more advanced imbalance-handling strategies and threshold analysis.
- Add richer model explainability and feature-attribution visualizations.
- Expand monitoring for prediction quality and data drift.
- Add authentication, role-based access and persistent assessment storage if required for a larger deployment.
- Evaluate the model on additional or institution-specific datasets before any real-world financial use.

6.10.6 Conclusion

LoanLens demonstrates a complete Machine Learning project workflow from problem definition and data analysis to model development, evaluation, visualization, API integration, frontend presentation, and cloud deployment. The project satisfies the academic workflow by implementing multiple classification algorithms and a scratch Logistic Regression implementation, while also providing a working deployed application for demonstrating ML inference.

Final Status: End-to-End LoanLens application completed, deployed, and production endpoints verified.

Appendix A — Main Production Endpoints

Method	Endpoint
GET	/api/health
POST	/api/predict
GET	/api/models
GET	/api/models/comparison
GET	/api/models/<modelId>
GET	/api/data/insights
POST	/api/simulate

Appendix B — Technology Stack

Layer	Technology
ML	Python, NumPy, scikit-learn
Inference	Flask
Backend/API Gateway	Node.js, Express, TypeScript
Rendering	EJS / Server-Side Rendering
Frontend	Vanilla JavaScript, HTML, CSS
Container	Docker
Repository	GitHub
Deployment	Render

Appendix C — Report Note

This consolidated document intentionally keeps the project's documented experiment records distinct: the academic notebook/PDF tuning results and the current deployed production artifact configuration are both represented rather than silently merged into a single historical experiment.